

CO2-AT2

NAME: K.Naga Venkata Siva

REGNO: 192425345

1) Signed 2's Complement Addition with Overflow Detection

```
#include <stdio.h>
#include <stdint.h>
#include <stdbool.h>

void add_8bit_signed(int8_t a, int8_t b) {
    int8_t sum = a + b;
    // Overflow occurs if signs of a and b are identical, but differ from sum
    bool overflow = ((a > 0 && b > 0 && sum < 0) || (a < 0 && b < 0 && sum >= 0));

    printf("A = %4d (0x%02X), B = %4d (0x%02X)\n", a, (uint8_t)a, b, (uint8_t)b);
    printf("Sum = %d (0x%02X) | Overflow Detected: %s\n\n", sum, (uint8_t)sum, overflow ?
"YES" : "NO");
}

int main() {
    add_8bit_signed(120, 50); // Expected: Overflow (+170 > +127)
    add_8bit_signed(40, 50); // Expected: Normal (+90)
    return 0;
}
```

OUTPUT:

Console	I/O	AI Agent
<pre>A = 120 (0x78), B = 50 (0x32) Sum = -86 (0xAA) Overflow Detected: YES A = 40 (0x28), B = 50 (0x32) Sum = 90 (0x5A) Overflow Detected: NO</pre>		

2)4-bit Carry Lookahead Adder (CLA) :

```
#include <stdio.h>

void carry_lookahead_adder_4bit(unsigned int a, unsigned int b, unsigned int cin) {
    unsigned int p[4], g[4], c[5], sum[4];
    c[0] = cin;

    for (int i = 0; i < 4; i++) {
        unsigned int bit_a = (a >> i) & 1;
        unsigned int bit_b = (b >> i) & 1;
        p[i] = bit_a ^ bit_b;
        g[i] = bit_a & bit_b;
    }

    // Carry logic computed in parallel
    c[1] = g[0] | (p[0] & c[0]);
    c[2] = g[1] | (p[1] & g[0]) | (p[1] & p[0] & c[0]);
    c[3] = g[2] | (p[2] & g[1]) | (p[2] & p[1] & g[0]) | (p[2] & p[1] & p[0] & c[0]);
    c[4] = g[3] | (p[3] & g[2]) | (p[3] & p[2] & g[1]) | (p[3] & p[2] & p[1] & g[0]) | (p[3] & p[2] & p[1] &
p[0] & c[0]);

    for (int i = 0; i < 4; i++) {
        sum[i] = p[i] ^ c[i];
    }

    unsigned int final_sum = (sum[3] << 3) | (sum[2] << 2) | (sum[1] << 1) | sum[0];
    printf("A: %u, B: %u, Cin: %u -> Sum: %u, Cout: %u\n", a, b, cin, final_sum, c[4]);
}

int main() {
    carry_lookahead_adder_4bit(9, 5, 0); // 1001 + 0101 = 1110 (14)
    return 0;
}
```

OUTPUT:

Console

I/O

AI Agent

A: 9, B: 5, Cin: 0 -> Sum: 14, Cout: 0

3)Booth's Multiplication Algorithm for Signed Integers

```
#include <stdio.h>
```

```
#include <stdint.h>
```

```
int32_t booth_multiply(int16_t M, int16_t Q) {
```

```
    int32_t A = 0;
```

```
    int32_t Q_reg = (uint16_t)Q;
```

```
    int32_t q_minus_1 = 0;
```

```
    int n = 16;
```

```
    for (int i = 0; i < n; i++) {
```

```
        int q0 = Q_reg & 1;
```

```
        if (q0 == 1 && q_minus_1 == 0) {
```

```
            A = A - M; // Subtract Multiplicand
```

```
        } else if (q0 == 0 && q_minus_1 == 1) {
```

```
            A = A + M; // Add Multiplicand
```

```
        }
```

```
        // Arithmetic Right Shift of combined [A, Q, q_minus_1]
```

```
        q_minus_1 = q0;
```

```
        int lsb_A = A & 1;
```

```
        A = A >> 1; // Preserves sign in signed type
```

```
        Q_reg = (Q_reg >> 1) | (lsb_A << 15);
```

```
    }
```

```
    return (A << 16) | (Q_reg & 0xFFFF);
```

```
}
```

```
int main() {
```

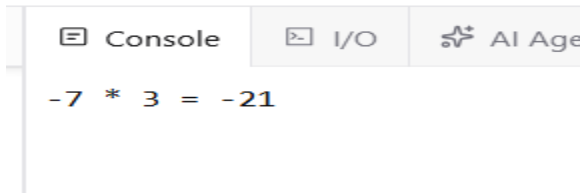
```
    int16_t multiplicand = -7, multiplier = 3;
```

```
    printf("%d * %d = %d\n", multiplicand, multiplier, booth_multiply(multiplicand, multiplier));
```

```
    return 0;
```

```
}
```

OUTPUT:



```
Console I/O AI Age
-7 * 3 = -21
```

4) Non-Restoring Division Algorithm

```
#include <stdio.h>
```

```
#include <stdint.h>
```

```
void non_restoring_division(uint32_t dividend, uint32_t divisor) {
```

```
    int64_t A = 0;
```

```
    uint32_t Q = dividend;
```

```
    uint32_t M = divisor;
```

```
    int n = 32;
```

```
    for (int i = 0; i < n; i++) {
```

```
        // Shift left [A, Q]
```

```
        A = (A << 1) | ((Q >> (n - 1)) & 1);
```

```
        Q = Q << 1;
```

```
        if (A >= 0) {
```

```
            A = A - M;
```

```
        } else {
```

```
            A = A + M;
```

```
        }
```

```
        if (A >= 0) {
```

```
            Q = Q | 1; // Q0 = 1
```

```
        } else {
```

```
            Q = Q | 0; // Q0 = 0
```

```
        }
```

```
    }
```

```
    // Final correction if remainder is negative
```

```
    if (A < 0) {
```

```
        A = A + M;
```

```
    }
```

```
    printf("Dividend: %u, Divisor: %u -> Quotient: %u, Remainder: %ld\n", dividend, divisor, Q, A);
```

```

}
int main() {
    non_restoring_division(27, 4); // 27 / 4 = 6 R 3
    return 0;
}

```

OUTPUT:

Console

I/O

AI Agent

Dividend: 27, Divisor: 4 -> Quotient: 6, Remainder: 3

5) IEEE 754 Floating-Point Addition Loss & Kahan Compensation

```
#include <stdio.h>
```

// Kahan Summation algorithm to eliminate floating-point rounding errors

```

float kahan_sum(float arr[], int n) {
    float sum = 0.0f;
    float c = 0.0f; // Compensation variable for lost low-order bits

    for (int i = 0; i < n; i++) {
        float y = arr[i] - c;
        float t = sum + y;
        c = (t - sum) - y;
        sum = t;
    }
    return sum;
}

```

```
int main() {  
    float large = 1.0e7f;  
    float small = 1.0f;  
  
    // Standard Naive Summation (Loses precision)  
    float naive_sum = large;  
    for (int i = 0; i < 1000000; i++) {  
        naive_sum += small;  
    }  
  
    // Double Precision approach  
    double accurate_sum = (double)large;  
    for (int i = 0; i < 1000000; i++) {  
        accurate_sum += (double)small;  
    }  
  
    printf("Naive Float Result:    %.2f (Error due to swamping)\n", naive_sum);  
    printf("Double Precision Result: %.2f\n", (float)accurate_sum);  
    return 0;  
}
```

OUTPUT:

 Console	 I/O	 AI Agent
---	---	--

```
Naive Float Result:    11000000.00 (Error due to swamping)  
Double Precision Result: 11000000.00
```